



BRL-CAD Python bindings

Personal Information:

Name	Abdullah Waleed Ahmed
Email ID	iwaleedahmed05@gmail.com
Location	Islamabad, Pakistan
Time Zone	UTC+05:00, GMT+05:00

Socials:

LinkedIn	https://www.linkedin.com/in/abdullah-waleed-ahmed-774890343/
Github	https://github.com/AWaleed-Ahmed

1. About Me

I am a second-year Software Engineering student at the National University of Sciences and Technology (NUST) with a strong interest in systems programming, software architecture, and open-source development. My work focuses on building reliable, maintainable systems and understanding how high-level software interacts with low-level infrastructure.

1.1 Open-Source Contributions to BRL-CAD

I have already contributed to BRL-CAD through several patches, which helped me become familiar with the project's architecture and development workflow:

1. Fixed a potential stack buffer overflow in `nastran-g.c` by replacing unsafe formatting with `snprintf`.
2. Resolved an in-memory database write failure in `librt`.
3. Fixed font persistence by syncing Tk named fonts to `mgd_default` before saving `.mgedrc`.

1.2 Prior Experience

1. Proficient in C++, Java, C, Python, JavaScript, and Dart.
2. Experienced with React, Node.js, and Flutter for web and mobile development.
3. Built full-stack systems, including TravelBestBets.com.
4. Developed AI/ML projects including crime-data regression models, graph-based recommendation systems, and a RAG-powered search engine.
5. Competed in multiple national hackathons, serving as a team lead and winning the DevCon 2026 Hackathon.
6. Strong experience with Git-based collaborative workflows, CLI tooling, and automation scripting.
7. Consistent focus on writing readable, maintainable, and modular software.

1.3 Recent Growth

1. Deepened practical knowledge of PyTorch and TensorFlow while preparing for open-source work.
2. Built hands-on experience with layered architecture: high-level APIs, exporter layers, and backend integration.
3. Improved debugging, testing, and performance optimization skills for complex systems.

4. Strengthened technical leadership by coordinating teams, delegating work, and delivering under strict deadlines.
5. Completed a web development internship at Arcana Info, gaining experience with professional team workflows and production delivery practices.

1.4 Value to Project

1. Low-level C/C++ debugging experience supports binding/adaptor reliability work.
2. Full-stack and architecture experience supports clean API and exporter design.
3. Proven BRL-CAD PR history shows familiarity with project standards, review expectations, and real-world maintenance needs.

2. Project Synopsis — BRL-CAD Python Bindings

I want to make BRL-CAD's Python workflow easier to use without redesigning the core C backend. I will be updating some parts but not rewriting the whole backend. The plan is to keep the architecture stable and improve the Python interface so users can model through the MOOSE path and reliably write and update .g files.

My first goal is reliability. I want database open and close behavior to be clean and consistent across the three modes we actually care about: read-only, writable, and in-memory database mode. And when something's invalid, the errors should be clear.

Second goal is correctness. I want to be able to support the main primitives sphere, box, cylinder, ELL, TGC and CSG operations like union, subtract, and intersect. And need to make sure transforms get written as a member matrix and that you can actually inspect them in MGED without any problems.

Third goal is that I want to add read access which provides a unified Python read entrypoint for supported object types with a stable return shape and deterministic errors for missing unsupported objects.

3. Benefits to the BRL-CAD Community

This work gives the BRL-CAD community one clear Python workflow based on MOOSE, with a stable C bridge that keeps BRL-CAD semantics intact and avoids C++ ABI binding risk. It improves reliability through explicit lifecycle rules and mode enforcement. It improves confidence in export results through matrix verification in MGED. It also adds a single read entrypoint for supported objects so users can inspect database contents from Python more easily. The result is a more usable and predictable Python experience for new users, while keeping the implementation incremental so that maintainers don't have a problem in inspecting any phase of the project.

4. System Overview

4.1 High-level idea of approach

The project standardizes on a MOOSE-based Python workflow for modeling and export. Users define primitives and CSG relationships in Python, and the export/read operations are routed through a stable bridge stack:

Python interface -> moose_rt3 adapter (ctypes) -> C bridge -> MOOSE C++ API -> BRL-CAD database core

The write path creates or updates BRL-CAD objects in .g files. The read path resolves supported object types and returns structured, deterministic results and transforms are stored as combination-member matrices, so primitive definitions remain reusable and can be validated in MGED.

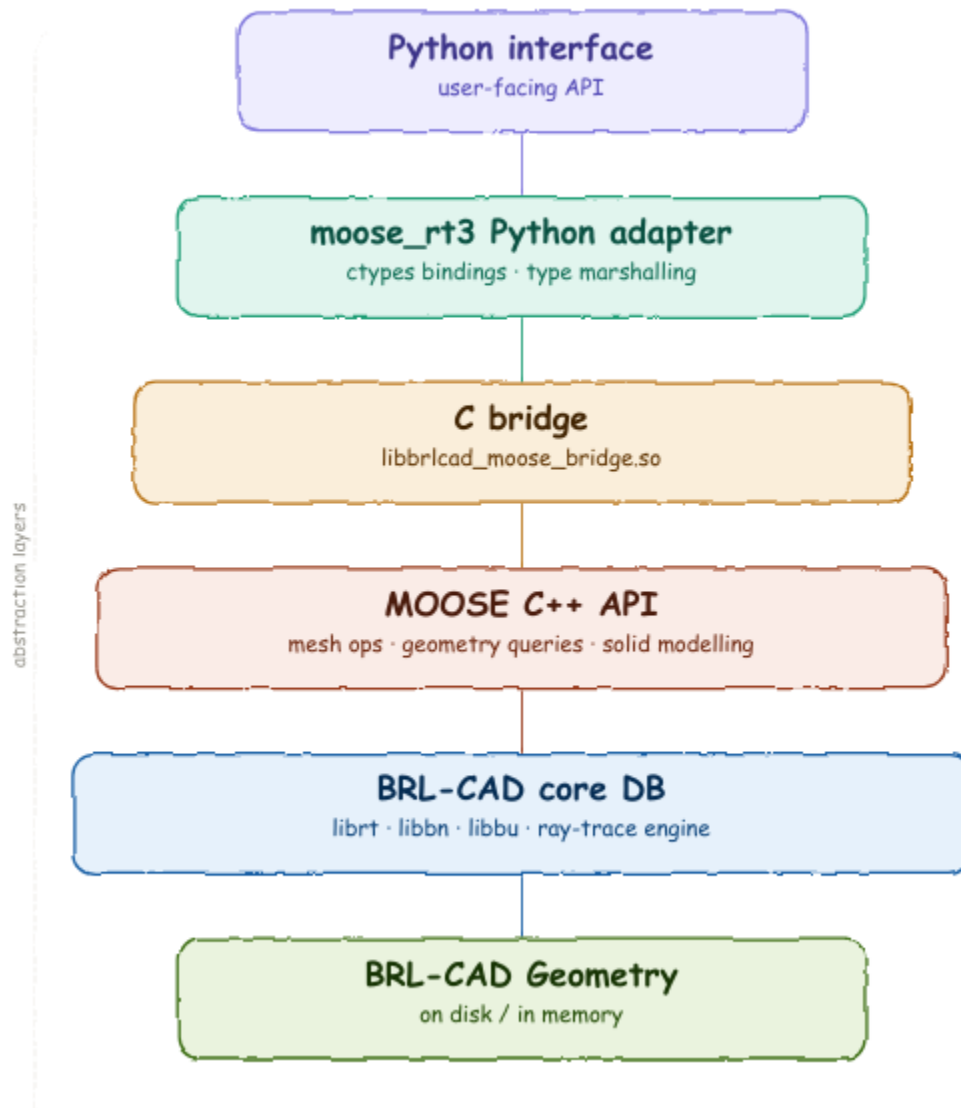
4.2 Layered architecture

Each layer has one responsibility:

- Python interface layer
 - User-facing API for creating primitives, building CSG relationships, setting attributes, and requesting read/write operations.
- moose_rt3 adapter layer (ctypes)
 - Python runtime adapter that defines C function signatures, marshals Python data to C-compatible types, and handles result/error conversion.
- C bridge layer
 - Stable C ABI boundary that isolates Python from compiler-specific C++ ABI details and translates adapter calls into MOOSE API operations.
- MOOSE C++ API layer
 - Core modeling/database API used for object creation, combination boolean logic, member-matrix transforms, attribute handling, and database access.
- BRL-CAD database backend layer
 - Executes geometry/database operations and persists .g output in read-only, writable, and in-memory modes.
- Verification workflow (not a stack layer):
 - Stored .g data is reopened for validation and inspection, including transform verification through combination-member matrices (for example in MGED).

5. System Architecture

5.1 Architecture Overview – Main Flow



5.2 Layered Design

To validate the feasibility of the proposed architecture, I developed a small experimental repository implementing an early version of the layered design described in this proposal.

Github repo: <https://github.com/AWaleed-Ahmed/python-bindings-prototype>

5.2.1 Python Interface

The Python interface is the entrypoint for modeling and database operations. It is designed as a clean, high-level API where users create primitives, build Boolean assemblies, and apply transforms through intuitive operations such as translate, rotate, and scale, without manually managing low-level matrix payloads. Internally, the engine materializes these operations into combination-member matrices to preserve BRL-CAD semantics using helper functions. The same interface supports attributes, mode-aware database lifecycle behavior, and unified read access with consistent result structures and deterministic errors. The goal is to keep BRL-CAD semantics visible while reducing user-side complexity.

5.2.2 In-Memory Model State

Before persistence, modeling intent is represented in Python-side state and then gets translated into backend operations. In my current approach, transform intent for composition is expressed as 16-value member matrices attached to combination members so primitive definitions remain reusable and placement is controlled at composition time.

5.2.3 Export and Update Flow

The export and update flow writes primitives and combinations to .g files then supports reopening the same database and applying additional changes. The transform matrices are written at the combination-member level and verified by reopening and reading the persisted results which confirms they are stored in the expected BRL-CAD composition layer rather than baked into primitive definitions.

5.2.4 Database Facade

The database facade provides one stable Python boundary for lifecycle, mode handling, write operations, and read operations. It enforces the three database modes required by scope: read-only, writable file, and in-memory. Invalid operations produce clear, deterministic errors. Supported reads return a consistent shape for object name, type, and attributes.

5.2.5 MOOSE_rt3 Python Adapter

The moose_rt3 adapter is implemented with ctypes and is responsible for exporting Python values into C-compatible types and calling the bridge API. It binds lifecycle, primitive creation, attribute operations, object read, and combination-member matrix functions. This keeps Python code simple and isolates low-level call details.

5.2.6 C-Bridge Layer

The C bridge is a thin, stable ABI layer between Python and MOOSE C++. It translates adapter calls into MOOSE operations and returns explicit status codes plus error messages. This layer exists specifically to avoid direct Python-to-C++ ABI coupling and to keep the integration portable across compiler environments and is intentionally kept close to MOOSE C++ functionality and naming. It is a translation boundary (C ABI exposure), not an alternate object model. This keeps behavior traceable to MOOSE semantics and simplifies long-term maintenance.

5.2.7 MOOSE and BRL-CAD Backend

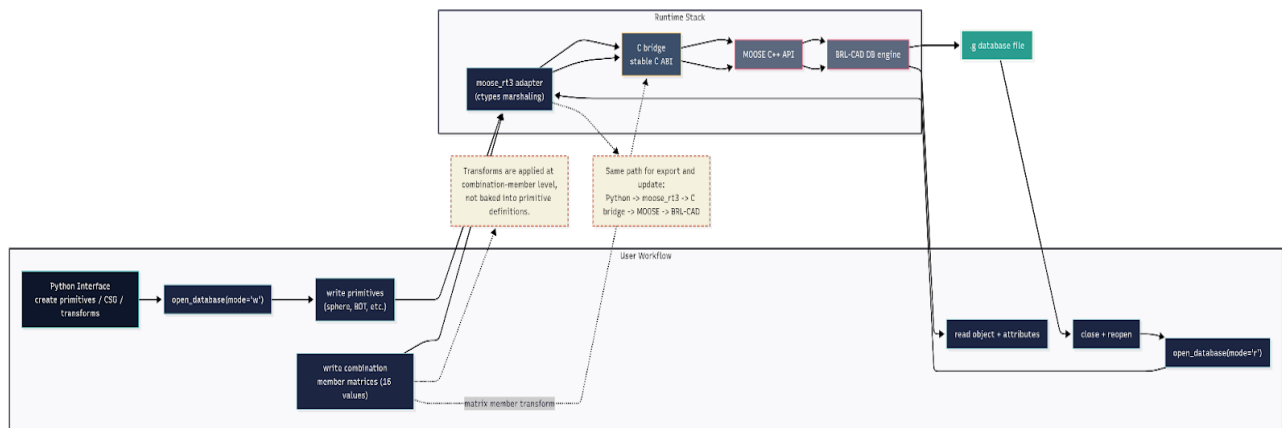
MOOSE provides the C++ API used for database/object operations, and BRL-CAD remains the external backend that executes geometry/database behavior and persists .g files, and in-memory database. No redesign of BRL-CAD core is required.

5.3 Transform Flow: Combination-Member Matrices

In this workflow, transforms are written at the combination-member level, which matches BRL-CAD composition semantics and keeps primitive definitions reusable. A single primitive can therefore be referenced multiple times with different placement matrices in different combinations.

During write/update, Python-side transform intent is represented as a 4×4 transform and passed as a flat 16-value matrix through the moose_rt3 adapter and C bridge to MOOSE, where it is attached to the corresponding combination member during combination construction.

Following is the user code flow:



5.4 Design Decisions

Foundation: Direct MOOSE C++ API Binding:

Our architecture will be routing all modeling operations through BRL-CAD's MOOSE C++ API via a thin ctypes bridge. Rather than constructing an intermediate Python object model, we translate user intent directly into MOOSE function calls. This preserves semantic fidelity at the cost of minimal translation overhead. One Python operation maps to one MOOSE operation which then maps to one .g representation which means that no intermediate interpretation layer introduces ambiguity or divergence. This design aligns user expectations with BRL-CAD's actual behavior and eliminates the long-term maintenance burden of keeping a separate modeling abstraction in sync with MOOSE evolution.

Combination-Member Matrices as Standard Transform Mechanism:

We represent all spatial relationships as transformation matrices stored on combination members. This choice reflects MOOSE's native assembly semantics: a primitive exists in canonical form, and its placement is determined by the member matrix within its containing combination. Primitives remain immutable and fully reusable; transforms are composition metadata. This approach is verifiable in MGED (members display their matrices directly), traceable through the API (each member's transform is explicit), and naturally persistent in .g format (BRL-CAD's native representation).

Three-Layer Architecture for Lifecycle Enforcement:

We implement Python interface → C adapter → MOOSE C++ API to enforce database mode semantics (read-only, writable, in-memory isolation). The C bridge translates Python calls into MOOSE operations while preventing mode violations by design. PyCapsule-based handles provide explicit lifetime ownership, reducing unsafe Python-C boundaries. This architecture directly serves our two deliverables: mode-enforced database lifecycle and correct transform propagation on export. It avoids architectural rewrite by leveraging proven infrastructure already proven at scale.

Binding Strategy: ctypes + Thin C Bridge:

For this cycle, the Python binding path uses ctypes over a thin C bridge instead of direct C++ bindings such as pybind11. This is a scope-control decision focused on delivery reliability. The C bridge provides a stable C ABI boundary between Python and the evolving MOOSE C++ layer, which reduces compiler and toolchain-coupling risk during core contract stabilization. It also keeps lifecycle, mode enforcement, and error behavior concentrated in one boundary that is easier to test deterministically.

Direct C++ bindings remain technically feasible, but introducing them in this cycle would increase build and packaging complexity while core behavior is still being finalized. Therefore we could treat pybind11 as a follow-on exploration after lifecycle, export, transform, and read contracts are stable.

6. Technical Approach

6.1 Scope and Outcomes

This implementation delivers three concrete outcomes:

- **Database mode enforcement:** read-only, writable file, and writable in-memory modes reject operations outside their contract with deterministic errors
- **Core geometry export with correct transforms:** sphere, box, cylinder, ellipsoid, and TGC primitives; union, subtract, intersect CSG combinations; and transform matrices stored on combination members (verified in MGED)
- **Unified read interface:** single query endpoint that resolves object type internally and returns documented structure for all supported types

Explicitly out of scope:

- Additional primitive families beyond sphere, box, cylinder, ellipsoid, TGC
- CSG optimization or flattening
- Incremental export (all-or-nothing per session)
- Advanced MOOSE features (volumetric weighting, region attributes, BoP tree operations)

6.2 Build Sequence

Work is sequenced in six ordered phases, each producing testable output:

1. Mode Enforcement

Write operations are gated at the database boundary by mode checks. Read-only mode raises errors on create/update. Writable file mode accepts writes and persists on close. Writable in-memory mode accepts writes without file output.

2. Export Traversal

The exporter walks the model tree depth-first: child geometry is serialized before parent combinations. Primitive nodes will create or resolve MOOSE objects by stable name. CSG nodes export children first, then emit the parent combination with the correct boolean operator (union, subtract, intersect).

3. Transform Accumulation and Member Attachment

Transforms are represented as 16-element C arrays (conceptually 4×4 matrices in row-major layout). When Python chains operations like `.translate(20,0,0).rotate_z(45)`, the exporter accumulates these into a single array by applying MOOSE matrix operations sequentially. The C adapter then passes this accumulated array to the MOOSE member create function, storing the transform metadata with the member record. The primitive definition remains unchanged; transforms are composition-scoped only.

4. Unified read API implementation

A single read entry point in the Database facade accepts an object name. The facade resolves the type internally, dispatches to the corresponding MOOSE reader, and returns a consistent result structure. Read is available in all three mode states. For file-backed workflows, close the read session and reopen the file

in writable mode to update transforms, add primitives, or restructure combinations. The unified read interface is essential because in-memory sessions have no persistent file to reopen; read-then-modify workflows require file mode.

5. Boundary validation

Harden MOOSE and facade boundaries for both read and write. Missing objects return not-found errors. Unsupported types return unsupported-type errors. Closed or invalid handles return lifecycle errors.

6. Lifecycle hardening and cleanup

Idempotent close, context-manager safety, and capsule-backed resource cleanup.

6.2.1 Implementation Illustration

Two implementation snippets are included to make the strategy concrete:

C Bridge: Combination Member Matrix Capture

```
int moose_comb_add_member(moose_comb_handle comb,
    const char* member_name, char operation,
    const double mat16[16]) {
    CombHandle* ch = reinterpret_cast<CombHandle*>(comb);
    if (!ch || !member_name) return fail(MOOSE_STATUS_INVALID_ARGUMENT,
        "comb/member_name must not be NULL");
    CombMember m;
    m.name = member_name;
    m.op = operation;
    m.has_matrix = (mat16 != nullptr);
    if (mat16) {
        for (size_t i = 0; i < 16; ++i) m.matrix[i] = mat16[i];
    }
    ch->members.push_back(m);
    g_last_error = "ok";
    return MOOSE_STATUS_OK;
}
```

Tree Construction with Explicit Member Transform Assignment

```

263
264     BRLCAD::Combination c;
265     c.SetName(comb_name);
266     if (ch->members.empty()) {
267         return fail(MOOSE_STATUS_INVALID_ARGUMENT, "combination has no members");
268     }
269
270     c.AddLeaf(ch->members[0].name.c_str());
271     if (ch->members[0].has_matrix) {
272         c.Tree().SetMatrix(ch->members[0].matrix.data());
273     }
274
275     BRLCAD::Combination::TreeNode tree = c.Tree();
276     for (size_t i = 1; i < ch->members.size(); ++i) {
277         tree = tree.Apply(map_op(ch->members[i].op), ch->members[i].name.c_str());
278         if (ch->members[i].has_matrix) {
279             BRLCAD::Combination::TreeNode right = tree.RightOperand();
280             if (right) {
281                 right.SetMatrix(ch->members[i].matrix.data());
282             }
283         }
284     }
285

```

Final Ideal API Design Goal

```

from brlcad import Scene, Sphere, Mesh, union

# User-facing API: no raw matrix16, no manual face-index wiring unless user wants Mesh.
scene = Scene()

sph = Sphere("sph", radius=12.5).translate(0, 0, 0)

# Mesh helper hides low-level BOT details; user provides high-level geometry intent.
mesh = (
    Mesh.from_points(
        "mesh",
        points=[
            (0.0, 0.0, 0.0),
            (10.0, 0.0, 0.0),
            (0.0, 10.0, 0.0),
            (0.0, 0.0, 10.0),
        ],
        topology="tetra" # helper decides faces internally
    )
    .translate(25, 0, 0)
    .rotate_z(15)
)

scene.add(sph)
scene.add(mesh)
scene.add(union("assembly", sph, mesh))

# Export path materializes transforms + member matrices internally.
scene.export("goal_scene.g")

# Unified read API remains simple.
with scene.open_database("goal_scene.g", mode="r") as db:
    print(db.read("assembly"))

```

6.3 Acceptance Criteria

Implementation is considered complete when all conditions below are true:

- Mode enforcement: read-only rejects all writes; writable file persists; writable in-memory does not create files
- Core export: sphere, box, cylinder, ellipsoid, TGC each export with correct parameters and appear in output .g file
- CSG export: union, subtract, intersect combinations map to correct BRL-CAD boolean operators
- Transform storage: member matrices are written to MOOSE, not primitive definitions; MGED displays matrices correctly when file is reopened
- Transform correctness: test scenes with translate, rotate, scale, and chained transforms produce expected numeric matrix values in export
- Export stability: repeated runs produce identical .g output
- Read API: single query point returns structured results for all supported types; handles missing objects, unsupported types, and closed handles with documented errors
- Read in all modes: read behavior functions identically in read-only, writable file, and writable in-memory states
- Lifecycle safety: context manager and explicit close paths clean up all native resources without leaks or double-frees
- Manual verification: open exported .g in MGED and confirm member matrices are present, accurate, and correctly ordered

6.4 Data Flow

1. Python Entry and Mode Contract

Users define primitives and CSG relationships in Python, then open a database session in one of three modes: read-only file, writable file, or writable in-memory. Mode is selected at session start and enforced for the full lifecycle of that handle. Read-only mode rejects all create and update operations at the boundary with deterministic errors. Writable file mode persists on close. Writable in-memory mode supports full construction flow but does not produce persistent file output.

2. Export Traversal

Export begins at the root of the model graph and proceeds depth-first so child geometry is emitted before parent combinations. This ordering guarantees that all referenced objects exist before they are attached into higher-level boolean structure.

3. Primitive Export

For each primitive node, the exporter resolves a stable object name and creates the corresponding MOOSE primitive when missing. Primitive definitions remain canonical and reusable; placement is not baked into primitive geometry.

4. Transform Accumulation and Member Attachment

For transformed nodes, the exporter accumulates operations in order into a single 16-value array representing a 4 by 4 matrix. That matrix is attached to the combination member record during combination construction. Transform ownership is therefore explicit at member scope, not primitive scope, matching MOOSE and BRL-CAD assembly semantics.

5. CSG Composition

For each CSG node, left and right subtrees are exported first, then combined using the mapped boolean operation: union, subtraction, or intersection. Member matrices are applied during combination tree construction so the stored assembly reflects both boolean structure and placement metadata.

6. Three-Layer Runtime Path

The runtime path is Python facade to C bridge ABI to MOOSE C++ API. The Python layer validates inputs and mode constraints, the C bridge validates handles and exported data, and MOOSE performs object creation and database writes. This single path removes model divergence and keeps behavior consistent from input to .g output.

7. Session Close and Persistence

Sessions close through explicit lifecycle handling. Writable file sessions persist to .g on close and reopen cleanly in MGED and other BRL-CAD tools. Writable in-memory sessions release state at close with no file persistence.

8. Unified Read Flow

Read requests enter through one facade endpoint by object name. The bridge resolves supported object data and returns a consistent structured response. Missing objects and unsupported types return deterministic, documented errors at the Python boundary.

7. Project Plan

7.1 Prioritization

- **Database lifecycle and mode enforcement**
Establish predictable open, use, and close behavior across read-only file, writable file, and writable in-memory sessions. Enforce mode contracts at the boundary so invalid operations fail immediately with deterministic errors.
- **Core write-path completeness through MOOSE bridge**
Complete and stabilize the required write surface in this phase: core primitive creation, combination creation, member insertion with matrix payload, and lifecycle-safe close semantics. Keep this path singular and explicit: Python facade -> C bridge ABI -> MOOSE C++ API.
- **Deterministic export flow**
Harden exporter traversal and naming so output is repeatable and children are emitted before parent combinations. Stabilize naming and emission logic so repeated runs generate consistent, reproducible structure.
- **Transform correctness through member matrices**
Accumulate chained transforms into one 16-value array representing a 4x4 matrix per member, then attach at combination-member insertion time. Preserve primitive canonical definitions and validate matrix values and operation order in exported output.

- **Unified read API for supported object types**

Provide one facade-level read endpoint by object name, with internal type resolution and a consistent response format. Return deterministic errors for missing objects, unsupported types, and invalid lifecycle state.

7.2 Feedback and Review Process

1. Weekly mentor sync with concise status/blockers.
2. Small PRs per feature area.
3. Early review on lifecycle and transform semantics.
4. Incorporate feedback before phase transition.

7.3 Parallel Work Strategy

1. Track A: tests/docs scaffolding.
2. Track B: implementation for current phase.
3. Continuous test runs to protect prior phases.
4. Short-lived branches to reduce merge risk.

8. Milestones & Timeline

8.0 Community Bonding (Pre-coding)

Deliverables:

- Align final scope with mentors and freeze non-goals.
- Validate test approach for mode behavior ,transform verification and unified read API.
- Finalize implementation checklist and review cadence.

Acceptance criteria:

- Mentor-approved scope statement is documented.
- Test plan and verification plan are agreed before coding starts.

8.1 Phase 1 (Weeks 2-3): Reliability Foundation

Deliverables:

- Make open/use/close behavior consistent across read-only file, writable file, and writable in-memory modes.
- Ensure close is idempotent and safe under repeated calls.
- Standardize invalid/closed-handle errors.
- Add lifecycle stress tests for repeated open/close and failure paths.

Acceptance Criteria:

- Lifecycle tests pass in all three modes.
- Double-close does not crash or leak resources.
- Invalid-handle errors are consistent and deterministic.

8.2 Phase 2 (Weeks 3-4): Mode Contract Enforcement

Deliverables:

- Enforce operation gating by mode at the facade boundary.
- Reject writes in read-only mode with clear errors.
- Allow writes in writable file and writable in-memory modes.
- Add focused tests for mode enforcement and edge cases.

Acceptance criteria:

- Read-only sessions reject all covered write paths.
- Writable sessions allow supported write operations.
- Mode tests pass with stable outcomes.

8.3 Phase 3 (Week 5-7): Core and Transform Export (Combination-Matrix Primary Path)

Deliverables:

- Harden depth-first exporter traversal for stable output.
- Finalize core primitive plus CSG export path.
- Accumulate transform chains into one 16-value array (4x4 matrix) per member.
- Attach the accumulated matrix at combination-member insertion in the MOOSE bridge path.
- Add integration tests for translate/rotate/scale and mixed chains on CSG members.

Acceptance criteria:

- Core primitive and CSG export tests pass end to end.

- Transform accumulation order is correct in automated checks.
- Member matrix attachment is confirmed on combination members.

8.4 Phase 4 (Weeks 8-9): Unified Read API

Deliverables:

- Implement unified read endpoint for supported object types.
- Define and document return schema.
- Add read tests for existing, missing, and unsupported objects.
- Validate read behavior across all three modes.

Acceptance criteria:

- Supported objects are readable through one API.
- Missing and unsupported errors are deterministic.
- Read tests pass across all three modes.

8.5 Phase 5 (Weeks 10-11): Verification and Tool Inspection

Deliverables:

- Add expected-value tests for known transform scenes.
- Perform manual BRL-CAD inspection of exported member matrices.
- Validate matrix ordering and placement in representative outputs.
- Resolve any mismatch between automated checks and tool inspection.

Acceptance criteria:

- Transform correctness suite passes.
- Manual inspection confirms matrix presence and expected ordering.
- Exported structure matches expected CSG topology and placement behavior.

8.6 Phase 6 (Weeks 12): Documentation and Final Hardening

Deliverables:

- Write focused documentation for this phase workflow.
- Document mode behavior, supported core primitives, CSG mapping, and transform ownership.
- Add minimal end-to-end examples for writable file and writable in-memory sessions.
- Run final regression pass for lifecycle, mode gating, and export stability.

Acceptance criteria:

- Documentation matches implemented behavior with no out-of-scope claims.
- Regression suite passes for lifecycle, mode gating, and export.
- Phase is ready for mentor review.

9. Deliverables

9.1 Concrete Outputs

Database reliability across three modes

Users can open a database in read-only mode to inspect existing files, writable file mode to create and save changes to disk, or writable in-memory mode to build geometry without touching the disk

Mode enforcement

If you try to create geometry in read-only mode, you get a clear error message. Writable modes allow you to create without errors. Error messages are the same every time, so you know what went wrong.

Stable Python facade over the MOOSE bridge path

A consistent interface for session management, primitive and combination writes, member-matrix attachment, and object reads, with explicit mode contracts and predictable error semantics.

Reliable core export pipeline

Depth-first export traversal (children before parents), stable naming behavior, and reproducible output for scoped primitive and CSG paths through the Python facade -> C bridge ABI -> MOOSE C++ API flow.

Correct transform handling

Transform chains are accumulated into one 16-value array (4x4 matrix) per member and attached during combination-member insertion. Primitive definitions remain canonical. Exported member matrices are inspectable in BRL-CAD tooling.

A unified way to read objects from the database

You can use one read function which figures out what type of object you asked for and returns the data in a consistent format. If the object is missing or not supported, you get a clear error.

Tests that verify everything works

The implementation includes tests for database behavior, export correctness, transform accuracy, and read functionality across all three modes.

Documentation

Clear guides on how to select a mode, build geometry, export, and read objects back from the database.

9.2 Scope Realism

1. Core primitive and CSG workflow prioritized.
Priority remains the core primitive and CSG export path with member-matrix transform semantics.
2. Incremental object coverage for read API.
Unified read API supports a bounded object subset in this phase, with explicit extension path for additional types.
3. Explicit non-goals are defined to keep the project scope focused.

10. Testing Strategy

Testing is done at unit, integration, and tool-inspection levels.

10.1 Mode contract tests

- Read-only mode rejects writes.
- Writable file mode persists and reopens correctly.
- Writable in-memory mode does not leave persistent file output.

10.2 Export correctness tests

- Validate scoped primitive creation paths through the Python facade and MOOSE bridge.
- Validate depth-first export ordering (children emitted before parent combinations).
- Validate CSG operation mapping to expected BRL-CAD boolean semantics (union, subtraction, intersection).

10.3 Transform correctness tests

- Validate chained transform accumulation into one 16-value array (4x4 matrix) per member.
- Validate operation-order correctness for representative translate, rotate, scale, and mixed-chain cases.
- Validate transform ownership at combination-member scope (not primitive-definition scope).

10.4 Manual BRL-CAD inspection

- Reopen representative exported outputs in MGED.
- Confirm member matrix presence, numeric ordering, and placement behavior against automated expectations.
- Resolve any mismatch between automated checks and tool-visible structure before phase sign-off.

10.5 Read API correctness tests

- Fetching existing supported objects returns expected structured data.

- Missing object requests return deterministic not-found errors.
- Unsupported object types return deterministic unsupported-type errors.
- Read behavior is mode-independent and works in read-only, writable file, and writable in-memory sessions.

10.6 Regression Gate

- Run combined regression for lifecycle safety, mode gating, core export, member-matrix persistence, and unified read behavior.
- Treat regression pass as required entry criterion for mentor review and phase closure.

11. Risks & Mitigation

11.1 Technical Risks

1. Transform order is applied incorrectly during export.
Mitigation: add unit tests with known translate/rotate/scale sequences and verify matrix values in MGED early, not at the end.
2. Due to Database lifecycle changes, open, close, and error paths may regress as mode contracts and bridge boundaries evolve
Mitigation: run repeated open/close stress tests for all three modes and keep phase gates that require lifecycle tests to pass before moving on.
3. Unified read API grows too broad too quickly which would destabilize error contracts and schema consistency.
Mitigation: start with a documented supported subset of object families and expand only after core behavior is stable.
4. MOOSE export has edge cases outside the supported workflow.
Mitigation: clearly document what is supported in this cycle and return explicit errors for unsupported cases instead of silent failure.
5. Python-side behavior drifts from BRL-CAD object semantics.
Mitigation: keep mapping tests that compare Python read/write behavior against BRL-CAD expectations for each supported object family.
6. Introducing a direct C++ binding path (for example pybind11) during this cycle could destabilize build and packaging while core behavior is still being finalized.
Mitigation: Keep ctypes + thin C bridge as the committed path for this cycle; evaluate direct C++ bindings only after lifecycle/export/read contracts are stable on a separate branch and after evaluation merge it with the main branch.

11.2 API Limitations

- Full BRL-CAD type parity is out of scope for one cycle.
- Some advanced update semantics may require phased rollout.
- Read API coverage is limited to a documented supported subset.
- Unsupported types remain explicit and return clear errors.

11.3 Fallback Plans

1. Prioritize shipping stable lifecycle behavior first.
2. Then ship stable core primitive/CSG export.
3. Keep unified read API limited to supported types if complexity rises.
4. Defer broader parity and advanced behaviors to stretch goals.

12. Post GSoC Plan

Beyond 175 hours:

1. Broader read/write parity across more BRL-CAD object families.
2. More complete update workflows for editing existing objects.
3. Performance profiling and optimization for large scenes.
4. Expanded tutorial notebooks and sample projects for MOOSE-first usage.
5. Higher-level validation tooling.
6. Advanced transform materialization options beyond matrix-member default where needed.
7. Evaluate a direct C++ binding path (for example pybind11) as a post-cycle exploration once core runtime contracts are stable.